

RESIDENCEVERTICAL

Partner Integration Guide

ResidenceVertical — Property Report API

One program: referral — link-only, or API-backed checkout links

Commission on every generated report, paid weekly by SEPA

Version 2.0 · September 2026

ResidenceVertical Partner API — integration guide (v2)

This guide is for developers and product owners at a partner company (real-estate portal, agency, broker network) who want to put ResidenceVertical premium property reports in front of their own users.

There is **one partner program: referral**. You send your users to **our** checkout, where they buy the report at the normal **50 lei** price; we handle payment, invoicing, delivery and support; and for every report that generates successfully you earn a commission, **paid to you weekly** by SEPA transfer. ResidenceVertical never charges a partner — money only ever flows from us to you.

You can integrate at either of two tiers, and both feed the same ledger, the same commission and the same weekly payout:

- **Link-only (zero code)** — you place your referral link (`/p/<your-slug>` , optionally pre-filled with the property address) on your site. No backend at all. §3.
- **API-backed (recommended if you have a backend)** — your backend mints a **checkout link** per lead with one API call (exact, server-side attribution and your own lead id on every conversion), and polls `GET /referrals` to learn which leads converted. §4.

If you only need the machine-readable contract, the OpenAPI document is served at `GET /api/partner/v1/openapi.yaml` on every environment (no authentication).

1. Overview — one program, two integration tiers

	Link-only (§3)	API-backed (§4)
What you build	A link on your site — nothing else	One server call per lead (<code>POST /checkout-links</code>), plus a small polling job
Who your user pays	ResidenceVertical — the normal 50 lei checkout	Same
Invoice to the end buyer	Ours (the standard buyer invoice)	Same
Report delivery	Our standard flow: delivery email + the report page	Same
What you earn	<code>commissionPct</code> × 50 lei per generated report	Same
Attribution	Stored in the visitor's browser for 30 days (last click wins)	Exact and server-side: the link was minted on your account; your <code>externalReference</code> comes back per conversion
How you learn of a sale	Your running totals on <code>GET /me</code>	<code>GET /referrals</code> per lead — poll it (§2.3)
Money direction	We pay you — weekly SEPA payout to your IBAN (§2.2)	Same

Ground rules, both tiers:

- **Only successfully generated reports earn.** A referral whose report fails terminally (the buyer is refunded by our standard flow), whose payment is refunded, or for which no report materialises, earns nothing (§2.1).
- **One account, one commission percentage.** `commissionPct` (default 15%, configured by ResidenceVertical per your agreement, visible on `GET /me`) applies to every referral, whichever tier it came through.
- **We never bill you.** There is no invoice from ResidenceVertical to a partner, no settlement, nothing to pay. The only money document between us is **your** commission invoice to us, for the amount we have paid out (§2.2).
- Days, weeks and months are counted in the `Europe/Bucharest` time zone.

Every partner receives an **API key** at onboarding. It authenticates the checkout-link mint (§4.1) and the tracking endpoints `GET /me` and `GET /referrals` (§4.2, §4.3); the buyer's purchase itself never involves it. Even a link-only partner keeps the key for tracking. Key-handling rules are in §6.

2. How the referral program works

Your site hands the user over to the ResidenceVertical checkout, the user completes the purchase like any direct buyer, and you earn your commission for every report that generates successfully. You never touch a payment, an invoice or a PDF.

The flow, end to end (either tier):

1. Your site sends the user to ResidenceVertical — through a checkout link you minted (§4.1), or through your static referral URL (§3).
2. Our page stores your attribution in the user's browser (valid **30 days**) and takes them into the normal report flow — under a discreet co-branding chip, "*Comandă prin partener: <your account name>*", on the order form and the payment summary, so the buyer sees the hand-off came from you without being interrupted by it.
3. The user pays the full **50 lei** on our checkout (card, via Stripe). We issue our standard buyer invoice and deliver the report exactly as for a direct customer — delivery email plus the interactive report page.
4. When the report reaches `generated`, your commission is **earned**: $\text{round}(5000 \times \text{commissionPct} / 100)$ bani per report — 750 bani = 7,50 lei at the default 15%.
5. Every Monday we close the previous week and **pay the week's earned commissions to your IBAN** in one SEPA transfer (§2.2). You invoice us for the commission.

```
sequenceDiagram
    autonumber
    participant U as End user (browser)
    participant P as Partner site / backend
    participant RV as ResidenceVertical
    U->>P: Clicks "Raport ResidenceVertical" next to a property
    alt API-backed (§4)
        P->>RV: POST /api/partner/v1/checkout-links (Bearer key, externalReference)
        RV-->>P: 201 { url: https://.../c/<token> }
        P-->>U: Redirect to url
    else Link-only (§3)
        P-->>U: Link to https://.../p/<your-slug>?street=...
    end
    U->>RV: Opens our checkout (address pre-filled, "Comandă prin partener" chip)
    U->>RV: Pays 50 lei – our invoice, our delivery email, our report page
    Note over RV: Report generated → your commission is EARNED
    P->>RV: GET /api/partner/v1/referrals (poll)
    RV-->>P: { status: "earned", externalReference, commissionCents }
    Note over RV,P: Monday: SEPA payout to your IBAN – you invoice us for it
```

See the program running before you build it

We host a working partner site on the test environment — a fictional property portal called *Portal Imobiliar Demo*, built on this exact API: <https://gamma.residencevertical.ro/partner-demo>

Each listing's button calls `POST /checkout-links` from that site's own backend, then hands the buyer to our checkout with the address pre-filled and the attribution attached; its "Conversiile mele" panel reads back live from `GET /referrals` (§4.2). It is the same flow as §4 below, and the same flow the bundled sample implements — just running against the real API.

The page itself is restricted to our team and to onboarded partners: tell us and we will enable your address. Your test key works against `/api/partner` from the moment you have it, with or without that.

2.1 What your user experiences — and what you earn

Your user is **our customer** for the transaction: our page, our 50 lei price, our card checkout, our invoice to them, our delivery email, our report page, our support and our refund handling. You are outside the payment and delivery path entirely, and **no buyer data flows back to you** — the referral ledger you can read (§4.2) deliberately carries no buyer email or any other personal data.

Commission is earned per report that reaches `generated`, at the `commissionPct` snapshotted when the purchase was attributed (a later percentage change never rewrites history). A referral that never turns into a generated report earns nothing: the report failed terminally (the buyer is refunded by our standard flow),

the payment was refunded, or no report materialised for the payment. You see each referral's state in `GET /referrals` (§4.2): `pending` → `earned`, or `void` for the cases that will never earn.

2.2 The weekly payout — we pay you

- A payout week runs **Monday 00:00 → Sunday 24:00, Europe/Bucharest**.
- **Every Monday** we close the previous week: all your commissions earned up to that Sunday that have not been paid yet are bundled into one payout. A week with nothing earned produces no payout row — nothing is lost, the next close picks up whatever is waiting.
- We pay the payout total to **your IBAN** by SEPA credit transfer. The remittance line names the period and your slug, so your bank statement is self-explanatory.
- The payout is **gross and B2B**: we withhold nothing. You issue **your commission invoice to ResidenceVertical** for the paid amount — the weekly statement email tells you the exact figure and period to invoice, so your accounting has a document trail on both sides.
- **No IBAN on file, no payout**. Earned commissions wait (they do not expire) until an IBAN and account-holder name are configured on your account — give them to us at onboarding, or whenever they change.
- **Refunds**. A buyer refund before the week closes voids that referral (it is simply not paid). A refund that lands after a payout was already paid is settled manually — we contact you and typically deduct it from the next payout.

Your running totals are always visible on `GET /me` (§4.3): `pending`, `earned-but-unpaid`, and everything paid out to date.

2.3 How you learn that a lead converted

You poll `GET /referrals` (§4.2). Referral conversions are **not** pushed to you — there is no webhook and no `referral.*` event today. A link-only partner without a backend simply reads the running totals on `GET /me` (§4.3) or asks us.

In practice that is a small job, because the money is settled weekly anyway:

- Poll every few minutes (or hourly, or nightly — pick what your funnel needs) and match `externalReference` against the leads you minted links for.
- `status` tells you where each one is: `pending` (paid, report generating) → `earned` (commission owed to you), or `void` (never earns).
- Nothing is lost by polling slowly. A conversion sits in the ledger indefinitely, and payment follows the weekly cycle in §2.2 regardless of when you read it.

If a push notification for referral conversions would change how you build, tell us — partners@residencevertical.ro. It is on our list and partner demand is what schedules it.

3. Tier 1 — the link-only integration (`/p/<your-slug>`)

No backend? The static referral URL is genuinely zero-code. Read it from `GET /me` (§4.3) — the `referral.referralUrl` field, e.g. `https://residencevertical.ro/p/agentia-exemplu`. Copy it as-is; do not construct variations of it yourself (the slug is assigned by us, and a mistyped slug silently earns you nothing).

How attribution works:

- Opening the link stores your partner slug in the visitor's **browser** and immediately continues into the normal report flow on our site. The attribution lives **30 days**; a newer partner link overwrites an older one (**last click wins**).
- The attribution is stamped on the payment **at purchase time**. If your user buys within the 30-day window — same visit or three weeks later — the report is yours.
- **The link can never break a sale**. An unknown or suspended slug, an expired attribution or a browser that cleared its storage simply loses the attribution — the user's checkout works regardless.
- Attribution requires the purchase to happen in the same browser the link was opened in; a user who switches devices between click and purchase is not attributed. Place the link where the buying intent is (next to the property, in the lead email), not only on a landing page.

What this tier does not give you. The whole attribution is your slug in the visitor's browser, so it is **device-local** and it carries **no lead id**. A link-only conversion lands in the ledger with `externalReference` and `checkoutLinkId` both `null` (§4.2): you see the money — your running totals on `GET /me` (§4.3) and the weekly payout (§2.2) — but not which listing, page or campaign earned it, and a user who clicks on their phone and pays on their laptop is never counted at all. That is the trade-off for writing no code. If you have a backend, mint checkout links instead (§4): same commission, same payout, but exact server-side attribution and your own `externalReference` back on every conversion.

Your buyer sees the same discreet co-branding chip as on the API-backed tier — *"Comandă prin partener: <your account name>"* — on the order form and the payment summary, so the hand-off is visible without interrupting the purchase. Our checkout reads your account name for that chip from the public partner lookup (§8.6); nothing on your side has to serve it.

Pre-filling the property address

You can append query parameters to the referral link and our landing pre-fills the report form with them — one less step for your user, and fewer typos in the address:

Parameter	Meaning
<code>street</code>	Street name, without the number (e.g. <code>Strada Turda</code>).
<code>number</code> (or <code>streetNumber</code>)	House/building number (e.g. <code>94</code> , <code>12B</code>).
<code>city</code>	Locality (e.g. <code>București</code>).
<code>type</code> (or <code>propertyType</code>)	<code>apartment</code> or <code>house</code> .
<code>postalCode</code>	Postal code (e.g. <code>011322</code>).

```
https://residencevertical.ro/p/agentia-exemplu?
street=Strada%20Turda&number=94&city=Bucure%C8%99ti&type=apartment
```

URL-encode the values (Romanian diacritics included). Everything is optional and the user can still edit it all — the parameters only pre-fill the form. Unknown parameters are ignored, and so is a `type` outside `apartment` / `house` (the user simply picks it).

Send type whenever you know it. Our form asks for the property type first and only reveals the address fields once it is chosen, so a link without `type` shows your user the type picker rather than the address you pre-filled. `street` and `city` are the minimum for any pre-fill to happen at all.

The parameters are a best-effort convenience; a **checkout link** (§4.1) pre-fills the same form server-side, attributes exactly, and adds per-lead tracking — prefer it whenever you have a backend to mint from.

4. Tier 2 — the API-backed integration

This is the tier to build if you have any backend at all: attribution does not depend on what the visitor's browser happens to have stored, and every conversion carries your own lead id back to you. Three calls cover it: mint a checkout link per lead (§4.1), poll the referral ledger (§4.2), and read your account (§4.3).

4.1 Checkout links — one call per lead

One server call mints a **checkout link** for one property; the returned `url` goes behind the button on your site ("Raport ResidenceVertical" next to the listing, or in the lead email). When the buyer opens it, our checkout starts with the address — and, if you sent one, the buyer's email — already filled in, under your co-branding chip.

Mint a link (server-side, with your API key — the key rules are in §6):

```
curl -sS -X POST "https://gamma.residencevertical.ro/api/partner/v1/checkout-links" \
-H "Authorization: Bearer $RV_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "address": { "street": "Strada Turda", "streetNumber": "94", "city": "București",
"postalCode": "011332" },
  "propertyType": "apartment",
  "externalReference": "lead-84213",
  "customer": { "email": "buyer@example.com" },
  "expiresInHours": 48
}'
```

Response **201 Created** :

```
{
  "checkoutLinkId": "pcl_Ab3dE9f1Gh2jK4mN6pQ8rS0tU1vW3xY5",
  "url": "https://gamma.residencevertical.ro/c/pcl_Ab3dE9f1Gh2jK4mN6pQ8rS0tU1vW3xY5",
  "expiresAt": "2026-08-20T10:00:00Z",
  "externalReference": "lead-84213"
}
```

`url` is the deliverable — hand it to the browser as-is (it carries no key; the opaque token is the whole link). `checkoutLinkId` is the same token, for your own records. The full field rules are in §8.1.

How a checkout link behaves:

- **Reusable until expiry, by design.** A link is never consumed by being opened: reloads, the back button or a second look days later must never brick the buyer. **Per-lead** uniqueness comes from your `externalReference`, not from a one-shot token — mint one link per lead and reuse it in that lead's context.
- **Expiry is yours to choose** (1 hour to 30 days, default 48 h). An expired link shows the buyer a calm message — *"Linkul de comandă nu mai este valid. Cere partenerului un link nou."* — with a button that continues to the plain order form, so it is never a dead end: the sale can still happen, only the pre-fill and the exact attribution are lost. Mint a fresh link when a lead re-engages after expiry.
- **Attribution is exact.** The link was minted on your account, so a purchase completed through it is attributed to you server-side, and the referral row copies the link's `externalReference` and `checkoutLinkId` (§4.2). The 30-day browser attribution window (§3) applies on top: a buyer who opens your link, leaves, and buys later within the window is still yours.
- Everything downstream is the standard referral economics: 50 lei paid to us, commission earned on `generated` only (§2.1), weekly payout (§2.2).

GET /checkout-links/{token}/resolve — the page calls this itself

Opening a checkout link, our page resolves the token through a **public** endpoint — no API key involved; the token is the whole credential:

```
GET /api/partner/v1/checkout-links/{token}/resolve
```

200 returns what the page pre-fills: { partnerSlug, partnerName, address{street, streetNumber, city, county, postalCode}, propertyType, customerEmail, expiresAt }. An expired link answers 410 `checkout_link_expired`; a token that never existed, 404 `not_found`. You never need to call it — it is documented because it is part of the contract (and the local mock implements it, §7, so you can watch the whole loop run before you have a key).

4.2 GET /api/partner/v1/referrals — your referral ledger

One row per attributed purchase. Newest first. `limit` ≤ 100 (default 20), `offset` default 0. Requires your API key (§6).


```
{
  "items": [
    {
      "referralId": "5b8e2f1a-7c3d-4e9b-a1f6-0c4d8e2b6a90",
      "createdAt": "2026-08-18T09:41:00Z",
      "status": "earned",
      "priceCents": 5000,
      "commissionCents": 750,
      "earnedAt": "2026-08-18T09:45:12Z",
      "paidAt": null,
      "externalReference": "lead-84213",
      "checkoutLinkId": "pcl_Ab3dE9f1Gh2jK4mN6pQ8rS0tU1vW3xY5"
    }
  ],
  "count": 1,
  "limit": 20,
  "offset": 0
}
```

Field	Meaning
referralId	The referral's id — quote it in support conversations.
createdAt	When the attributed purchase was confirmed.
status	pending (paid, report generating) → earned (report generated — commission owed to you), or void (never earns: the report failed, the payment was refunded, or no report materialised).
priceCents / commissionCents	The report's list price (5000 = 50 lei) and your commission on it, snapshotted at attribution time.
earnedAt	When the report reached generated; null for pending / void.
paidAt	When the weekly payout containing this referral was paid; null until then. status stays earned — paidAt is the paid marker.
externalReference	Your own lead id, copied from the checkout link (§4.1) the buyer converted through — the per-lead conversion tracking : match it against the ids you minted with. null when the buyer came through the link-only tier (§3).
checkoutLinkId	The checkout link (pcl_...) that attributed this purchase; null for link-only attributions.

count is the number of items **in this page** (not the total across all pages) — to walk your history, increase offset by limit until a page comes back with fewer than limit items. Store the referralId values you have already seen so a re-poll is cheap.

Deliberately absent: any buyer data. The buyer is our customer; you get the money trail, not the person.

4.3 GET /api/partner/v1/me — your account and running totals

GET /me (§8.4) carries a referral object for every account:

```

"referral": {
  "referralUrl": "https://gamma.residencevertical.ro/p/agentia-exemplu",
  "commissionPct": 15,
  "pendingCents": 750,
  "earnedUnpaidCents": 1500,
  "paidCentsAllTime": 2250
}

```

Field	Meaning
referralUrl	The link-only URL to put on your site (§3), on this environment's host. Checkout links (§4.1) are minted per lead instead of read from here.
commissionPct	Your percentage — the same value as the top-level field.
pendingCents	Commission on attributed purchases whose report is still generating (bani). Not earned yet.
earnedUnpaidCents	Earned commission waiting for its weekly payout (or sitting in a payout not yet paid).
paidCentsAllTime	Total commission already paid out to you, ever.

4.4 Testing on the test environment

On the test environment the checkout pages (`/c/<token>` and the `/p` landing included) sit behind our team access wall (unlike `/api/partner`, which is open), so an end-to-end referral test on `gamma.residencevertical.ro` is something we run **together** at onboarding. Everything under `/api/partner` works with your test key at any time: minting checkout links, the public resolve, and the tracking endpoints (`GET /me`, `GET /referrals`). The local mock (§7) covers the whole loop — mint, resolve, a stand-in `/c/<token>` landing, and canned referral data — so you can build your integration and reconciliation first.

5. Environments

Purpose	Base URL	Notes
Integration / testing	<code>https://gamma.residencevertical.ro</code>	Test environment. Data may be reset without notice and the environment may be unavailable during maintenance windows. Use it for building and testing your integration.
Production	<code>https://residencevertical.ro</code>	Enabled for your account after go-live sign-off with ResidenceVertical.

API keys are **environment-specific**: a key issued for the test environment starts with `rvp_test_` and a production key starts with `rvp_live_`. A test key never works in production and vice versa. Partner accounts and referral ledgers are also per environment.

All examples use the test base URL. Every path is prefixed with `/api/partner/v1`.

6. Authentication and key handling

Every endpoint except the three public ones — `GET /openapi.yaml` (§8.5), the checkout-link resolve (§8.2) and the partner name lookup (§8.6), all three called by our own pages or your tooling rather than by your backend — requires:

```
Authorization: Bearer rvp_test_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Rules:

- **Server-side only.** The key must live in your backend's secret store. Never embed it in browser JavaScript, mobile apps, HTML, public repositories or client-side config — anyone holding the key can mint checkout links on your account and read your referral ledger.
- **HTTPS only.** Both environments are HTTPS; plain HTTP is not served.
- **Rotation.** ResidenceVertical can issue several keys per account (for example one per server or per staging/production deployment). Ask for a new key, switch your servers over, then ask us to revoke the old one — there is no downtime.
- **Revocation.** A revoked or expired key returns `401 unauthorized` immediately. If you suspect a leak, contact support at once so we can revoke it.
- The full plaintext key is shown to ResidenceVertical staff **once** when it is created and handed to you through a secure channel; we only store a hash and the first 16 characters (`rvp_test_AbCd123`) for display. Keep your own copy safe.

Every response, success or error, carries an `X-RV-Request-Id` header. Log it next to your own request id — support will ask for it.

7. Quick start (curl)

Prove the key works and read your referral link:

```
curl -sS "https://gamma.residencevertical.ro/api/partner/v1/me" \
-H "Authorization: Bearer $RV_API_KEY" | jq '.referral'
```

Mint a checkout link for a lead (§4.1):

```
curl -sS -X POST "https://gamma.residencevertical.ro/api/partner/v1/checkout-links" \
-H "Authorization: Bearer $RV_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "address": { "street": "Strada Turda", "streetNumber": "94", "city": "București",
"postalCode": "011332" },
  "propertyType": "apartment",
  "externalReference": "lead-84213"
}' | jq -r '.url'
```

Put the printed `url` behind your button. Later, see which leads converted (§4.2):

```
curl -sS "https://gamma.residencevertical.ro/api/partner/v1/referrals?limit=50" \
-H "Authorization: Bearer $RV_API_KEY" | jq '.items[] | {externalReference, status, commissionCents}'
```

Reference integration and mock server — start before you have a key

A complete, runnable Node.js integration lives in the `sample/` directory of [this documentation repository](#): a demo property page, the partner backend behind it, and **a local mock of this API**.

```
node mock-rv-api.js    # a local stand-in for this API, on :4010
node server.js         # the demo site + its backend, on :4000 → http://localhost:4000
```

Zero dependencies — Node.js ≥ 20 built-ins only, no `npm install`. The mock speaks the contract described here: Bearer auth, the error envelope and every `error.code`, `X-RV-Request-Id`, **server-minted checkout links** (`POST /checkout-links`, the public resolve and a stand-in `/c/<token>` checkout landing with the co-branding chip), and a canned referral ledger on `GET /referrals` whose money agrees with the `/me` `referral` block (§4.2, §4.3), so you can build your integration and your payout reconciliation before a single real referral exists. `MOCK_CHECKOUT_LINK_TTL_SECONDS` compresses a link's lifetime into a few seconds so you can see the expired state your buyer would get.

When the key arrives, point the same code at this environment with two environment variables (`RV_API_BASE_URL`, `RV_API_KEY`); no code changes.

8. Endpoint reference

All requests and responses are JSON (`Content-Type: application/json`) except the OpenAPI document. Timestamps are ISO-8601 UTC (`2026-08-18T10:00:00Z`). Money is expressed in **cents of RON** (bani): `5000` = 50 lei.

8.1 `POST /api/partner/v1/checkout-links` — mint a checkout link

Headers: `Authorization`, `Content-Type: application/json`.

Field	Required	Rules
<code>address.street</code>	yes	Street name without the number (e.g. <code>Strada Turda</code> , <code>Bulevardul Unirii</code>).
<code>address.streetNumber</code>	yes	House/building number, e.g. <code>94</code> , <code>12B</code> , <code>229-231</code> .
<code>address.city</code>	yes	Locality, e.g. <code>București</code> , <code>Cluj-Napoca</code> , <code>Chiajna</code> .
<code>address.county</code>	no	County (județ).
<code>address.postalCode</code>	no	Postal code.
<code>propertyType</code>	yes	<code>apartment</code> or <code>house</code> .
<code>externalReference</code>	no — but send it	Your lead/order id, ≤ 128 chars. Echoed on the link and, once the buyer converts, on that referral's row in <code>GET /referrals</code> (§4.2) — this is your per-lead conversion tracking .

Field	Required	Rules
customer.email	no	RFC-shaped. Stored with the link, used to pre-fill the buyer's email on our checkout, and returned by the link's <code>resolve</code> endpoint (§8.2) to the browser that opens the link. Minting a link sends no email. Only include it where you have a lawful basis to share it — see §10.1.
expiresInHours	no	Integer, 1..720 (up to 30 days); default 48 .

No `coordinates` field — the buy flow geocodes the address exactly as it does for a direct buyer.

Response **201 Created**: { `checkoutLinkId`, `url`, `expiresAt`, `externalReference` } — the example and the link's behaviour are in §4.1. Validation failures answer **400 validation_error** with per-field messages (§9).

8.2 GET /api/partner/v1/checkout-links/{token}/resolve — public resolve

Public — no API key. Called by our checkout page when a buyer opens a link; the token is the whole credential. **200** with { `partnerSlug`, `partnerName`, `address`, `propertyType`, `customerEmail`, `expiresAt` }; **410 checkout_link_expired** after expiry; **404 not_found** for a token that never existed. You never call it yourself (§4.1).

8.3 GET /api/partner/v1/referrals?limit=&offset= — your referral ledger

Documented in §4.2: newest first, `limit`/`offset` pagination in the { `items`, `count`, `limit`, `offset` } envelope, one row per attributed purchase with `status` `pending` / `earned` / `void`, `paidAt` set once its weekly payout is paid, and — for purchases attributed through a checkout link — your `externalReference` plus the `checkoutLinkId`. No buyer data is exposed.

8.4 GET /api/partner/v1/me — your account

```
{
  "partnerId": "...",
  "name": "Agenția Exemplu",
  "slug": "agentia-exemplu",
  "environment": "test",
  "commissionPct": 15,
  "reportPriceCents": 5000,
  "currency": "RON",
  "referral": {
    "referralUrl": "https://gamma.residencevertical.ro/p/agentia-exemplu",
    "commissionPct": 15,
    "pendingCents": 750,
    "earnedUnpaidCents": 1500,
    "paidCentsAllTime": 2250
  },
  "dailyReportCap": null,
  "reportsToday": 0,
  "webhookConfigured": false,
  "usageThisMonth": { "requested": 0, "generated": 0, "failed": 0, "commissionCents": 0 }
}
```

`environment` is `test` or `live`. `reportPriceCents` / `currency` are the list price your buyers pay (50 lei). Use this endpoint to verify a new key, to read your referral link, and to show your own dashboards from the `referral` block (§4.3).

The last four fields — `dailyReportCap`, `reportsToday`, `webhookConfigured` and `usageThisMonth` — belong to the retired partner-generated-report API (§13). They are kept on the wire for compatibility, are always empty for a referral account, and can be ignored.

8.5 GET /api/partner/v1/openapi.yaml — OpenAPI 3 document

Public, no authentication, `Content-Type: text/yaml`. Generate clients from it or import it into your API tooling.

8.6 GET /api/partner/v1/partners/{slug}/public — public partner name lookup

Public — no API key. Our checkout calls it when a buyer arrives through your `/p/<your-slug>` link (§3), to render the co-branding chip. `200` returns `{ slug, name }` — your account name and nothing else; no commercial figure is exposed on a public surface. An unknown slug and a **suspended** account both answer `404 not_found`, so the endpoint cannot be used to enumerate partners or to read the state of one. You never call it yourself; it is documented because it is part of the public contract and appears in the OpenAPI document (§8.5).

9. Error handling

Errors use one JSON shape:

```
{ "error": { "code": "validation_error", "message": "Request validation failed:  
address.streetNumber is required", "fields": { "address.streetNumber": "address.streetNumber  
is required" } } }
```

`fields` is present only on validation errors; `message` joins every field message with `;` (e.g. Request validation failed: address.streetNumber is required; `propertyType` must be one of: apartment, house). Branch on `code` (and the `fields` keys), never on the text of `message`.

HTTP	code	Meaning	What to do
400	validation_error	Body failed validation; details in <code>message</code> / <code>fields</code> .	Fix the request. Do not retry unchanged.
401	unauthorized	Missing, malformed, unknown, revoked or expired key.	Check the <code>Authorization</code> header and the environment (<code>rvp_test_</code> vs <code>rvp_live_</code>). Contact support if the key should be valid.
403	partner_suspended	The key is valid but the account is suspended.	Contact ResidenceVertical.
403	billed_generation_retired	You called the retired partner-generated-report endpoint (§13).	Sell through checkout links instead (§4.1).
404	not_found	Unknown resource, or one that belongs to another account.	Check the id.
410	checkout_link_expired	A checkout link was resolved after its expiry — answered by the public resolve endpoint to our own checkout page (§4.1), never to a call you make with your key.	Nothing on your side fails: the buyer sees a calm state that points back at you. Mint a fresh link for the lead.
429	(no JSON body)	Platform per-IP rate limit at the edge (where the environment applies one).	Back off (honour <code>Retry-After</code> if present) and retry; see §11.
503	partner_api_disabled	The Partner API is switched off on this environment.	Retry later; contact support if it persists.

Any 5xx without a JSON body comes from infrastructure in front of the API (load balancer, edge). Treat it as transient and retry with backoff. Minting a checkout link is safe to retry: a retry simply mints a second link for the same lead, and either one attributes the sale to you with the same `externalReference`.

10. Data you send (and what happens with it)

- **Address.** `street` (name only), `streetNumber` and `city` are required. The more precise the address the better the report; include `postalCode` and `county` when you have them. The number matters — much of the report (developer attribution, building permits, seismic registry) is resolved at building level, so a street without a number produces a much weaker report for your buyer. Our checkout geocodes the address for the buyer exactly as it does for a direct customer.
- **propertyType.** `apartment` or `house`. Sending the wrong type changes the analysis vocabulary and checks (e.g. a house report covers land/cadastral topics that an apartment report does not). The buyer can still correct it on our form.
- **externalReference.** Your lead/order id, echoed on the link and on the referral row the conversion produces — the easiest way to correlate our ledger with your data.
- **customer.email (optional).** Only used to pre-fill the email field on our checkout so the buyer types less. Minting a link sends nothing; the buyer receives our standard delivery email only once they have bought.

10.1 Personal data

Where you send us an end user's personal data — today only `customer.email` on a checkout link — both parties act as **independent controllers** for their own processing. By sending it you warrant that you have a lawful basis to do so and that you have informed your users that their details are shared with ResidenceVertical to prepare their order.

We use it to pre-fill the checkout and, once the buyer has bought, to deliver the report and meet our accounting obligations. We do not sell it or use it to market to your users.

Note the one place it is readable outside the API: a checkout link's `customer.email` is returned by the public `resolve` endpoint (§8.2) to whoever opens that link, so that our checkout can pre-fill it. Treat a checkout-link URL as something to send only to the buyer it was minted for.

A data-sharing agreement covering roles, retention and deletion requests is executed as part of partner onboarding, before any live traffic. Our privacy policy: <https://residencevertical.ro/privacy>. Questions: partners@residencevertical.ro.

11. Rate limits

Depending on the environment, the platform may apply per-IP limits at the edge (on the order of 40 requests/second across all endpoints). Normal integrations never approach this: one mint per lead plus a referral poll every few minutes is a handful of requests a day. If you do get a `429`, honour `Retry-After` when present and back off.

Mint one checkout link per lead and reuse it in that lead's context (§4.1) rather than minting on every page view — links are reusable until they expire.

12. Support and escalation

- Support: support@residencevertical.ro.

- Always include: environment (test/live), your partner slug, the `externalReference` or `referralId` concerned, the `X-RV-Request-Id` of the failing call, the timestamp (UTC), and the HTTP status + `error.code` you received.
- Key leaks, suspected fraud or an urgent production outage: say so in the subject line and we will revoke/rotate keys and investigate with priority.
- Roadmap items not shipped yet: a push notification for referral conversions (§2.3), a partner self-service portal, and an embeddable JavaScript widget. Tell us if any of these would unblock your integration.

13. Partners who used the retired generation API

Until v2.0 the API also offered a mode in which a partner generated reports server-to-server on its own account and was invoiced for them weekly. **That mode is retired and is no longer offered:**

ResidenceVertical does not bill partners, and reports are sold only through our checkout, with the partner earning a commission (§1).

A call to the retired generation endpoint now answers `403 billed_generation_retired` (§9). Nothing produced before the retirement has been deleted, and the referral endpoints in this guide work on the same account with the same key.

If you integrated against the old mode, write to partners@residencevertical.ro. We will confirm what your account can still read of its own history and walk you through switching your button to checkout links (§4.1) — it is one call, and the commission replaces what you used to be invoiced for.

14. Changelog

Version	Date	Notes
v2.0	2026-09-02	One program: referral. Partner-billed report generation ("API mode") is retired — ResidenceVertical never collects money from a partner. That mode's generation call now answers <code>403 billed_generation_retired</code> and nothing it produced has been deleted (§13). The guide is restructured around the one program and its two integration tiers — link-only (§3) and API-backed checkout links + referral polling (§4) — and now also documents the public partner name lookup behind the checkout co-branding chip (§8.6), which has been live since v1.4. No referral endpoint or field changed.
v1.4	2026-08-21	Checkout links + co-branding. <code>POST /api/partner/v1/checkout-links</code> (§4.1) mints a server-attributed <code>/c/<token></code> checkout URL for one property — the buyer lands on our checkout with the address (and, optionally, their email) prefilled; links are reusable until expiry (<code>expiresInHours</code> 1..720, default 48 h); a public <code>GET /checkout-links/{token}/resolve</code> backs the page (new error code <code>checkout_link_expired</code>). Per-lead conversion tracking: <code>GET /referrals</code> items now carry <code>externalReference</code> + <code>checkoutLinkId</code> , copied from the link at attribution time (§4.2). Both tiers show a discreet co-branding chip on the checkout — <i>"Comandă prin partener: <name>".</i> The <code>/p/<your-slug></code> link stays as the zero-code option (§3). Additive only.

Version	Date	Notes
v1.3	2026-08-21	Referral program. You link your users to our checkout via <code>/p/<your-slug></code> (with optional <code>street / number / city</code> prefill parameters), the user pays the normal 50 lei on our checkout with our standard invoice and delivery, and you earn <code>commissionPct</code> per generated report — paid to you weekly by SEPA transfer to your IBAN (gross, B2B; you invoice us for the commission). New: the <code>referral</code> block on <code>GET /me</code> (§4.3) and <code>GET /api/partner/v1/referrals</code> (§4.2); the weekly statement email. Additive only.
v1.2	2026-08-21	Weekly settlements for the (since retired) partner-generated-report mode; <code>GET /api/partner/v1/settlements</code> .
v1.1	2026-08-20	Report web view for partner-generated reports (<code>viewUrl</code> , <code>POST /reports/{id}/view-link</code> , token-authenticated page endpoints).
v1.0	2026-08-18	Initial release: partner-generated reports (<code>create/status/list/pdf/me</code>), idempotency, signed webhooks, daily cap, OpenAPI document.